

Multi-agent matchmaking

L Foner and I B Crabtree

Many important and useful applications for software agents require multiple agents on a network that communicate with each other. Such agents must find each other and perform a useful joint computation without having to know about every other such agent on the network. This paper describes Yenta, a matchmaker system designed to find people with similar interests and introduce them to each other. It describes how the agents that make up the matchmaking system can function in a decentralised fashion, yet can group themselves into clusters which reflect their users' interests. These clusters are then used to make introductions or allow users to send messages to others who share their interests. The algorithm uses referrals from one agent to another in the same fashion that word-of-mouth is used when people are looking for an expert. A prototype of the system has been implemented, and the results of its use are presented.

1. Introduction

Software agents are computer programs which attempt to perform some set of tasks autonomously for their users in a trustworthy, personalised fashion. They can be either manually programmed by the user, or use techniques from machine learning to discover how the user does some task and gradually automate it. Examples include mail filtering programs, which learn or are told whose mail is valued and whose is not [1, 2], meeting-scheduling programs, which learn or are told when and with whom to schedule meetings and how flexible to be in negotiating (with other agents) for times depending on who else is in the meeting [3], and so forth. Many software agents are even designed to be primarily entertaining, perhaps with ancillary practical or informative goals [4, 5].

Other agents take more initiative; they actively inform the user when they find items that match the user's known interests. Often, such agents may not understand the domain of interest directly, but are instead facilitators that can find other people who understand the domain better who can advise. Automated collaborative filtering, in which users with similar tastes are matched up, is used in systems such as Webhound [1] or MORSE [6].

While the two agents above match up users' tastes to make recommendations, their focus is not explicitly to matchmaking users and introducing them to each other. The research described in this paper is focused on introducing users who are interested in similar topics. There are a number of reasons why one might want to do this:

- people are often working on similar projects without realising it — be it two people down the hall from each other re-inventing the same wheel, or two doctors both doing research on similar cases but having no idea that both of them are studying the same domain,
- it is often the case that people need to find an expert in some field, but finding such an expert can be difficult and time-consuming — those who are not well 'plugged-in' via word-of-mouth can find this even more difficult,
- in a large organisation, there are likely to be many initiatives that make use of the same underlying methods and techniques — project directories and job titles will be of no use in identifying them but knowledge of people with similar background would be very useful.

1.1 Why having multi-agent systems helps

Many currently implemented agents use a centralised architecture, in which one agent serves either one or many users. A centralised architecture has its advantages; for example, if there is no effective way for peers to find each other, a centralised solution may be the only workable solution. Unfortunately, there are problems with a centralised architecture:

- scaling such an architecture to large numbers of users is difficult — in systems which must correlate user interests (for example, Shardanand and Maes [7]), straightforward approaches to this problem generally require a quadratic-order complexity,
- if the system requires either high availability (due to constant demand for its services) or high trustability (because it handles potentially sensitive information, such as personal data), a centralised server provides a single point where either accidental failure or deliberate compromise can have catastrophic consequences.

For these reasons and others, many foreseeable future applications for software agents will involve large numbers of agents interacting with each other. Users may have a number of agents operating on their behalf, and agents of any particular user may have to communicate with other agents elsewhere on the network in order to share information.

1.2 Why multi-agent systems are hard to build

While decentralised, multi-agent systems have several important advantages, one of the largest problems with them is how agents are supposed to find each other. Each agent should not have to know about (and, indeed, probably cannot know about) every other agent, user or resource on the network. Instead, some mechanism by which agents may locate only the useful agents on the network must be arranged.

There are several relatively straightforward approaches that have been used in other networked systems. For example, hierarchical organisation of the entities, as is done with resource records in the Internet domain-name system [8] or with newsgroup topics in the Usenet [9], can help to reduce the inherently quadratic problem into a logarithmic one. However, such approaches depend on some inherent organisational principle that is established in advance, which is neither always optimal nor always convenient; for example, consider the number of crossposted Usenet articles, a clear indication that single-inheritance hierarchies are not necessarily a good match to the underlying topic space.

This research focuses on the problems of a matchmaking service, one designed to find groups of people with similar interests and bring them together to form coalitions and interest groups. This paper does not explicitly deal in romantic matchmaking between users, for many reasons — the most obvious being that shared interests do not necessarily mean that two people are romantically compatible. The intended scale of the matchmaking is that of the entire Internet, an environment in which there are

potentially millions of users and millions of agents corresponding to them. The domain and the large number of agents present difficult co-ordination problems, such as:

- there is no obvious *a priori* hierarchy by which to organise the agents (Why would any one person's interests be at the top of any hierarchy? How would we know whom to pick, anyway?),
- asking other agents at random resembles diffusion in a gas and is extremely slow — it means each agent could be required to ask every agent on the network, guaranteeing a solution that scales poorly,
- a centralised approach runs into the problems mentioned above of quadratic complexity, and also is subject to single-point-of-failure problems if the central system either fails or is compromised — an important point for an application handling potentially sensitive data.

1.3 Finding the right cluster of peer agents — the core idea

To address these problems, this research considers an overall organisation which borrows ideas from computational ecology [10], in which agents have only local knowledge, but self-organise into larger units. The core ideas in the approach taken here are to:

- compare the agents' information in a peer-to-peer, decentralised fashion,
- use referrals from one agent to another and an algorithm resembling hill-climbing to find other, more appropriate agents when searching for relevant peers,
- build clusters or clumps of like-minded agents,
- use these clusters of similar or like-minded agents (whose users therefore share similar interests) to introduce users to each other and enable cluster-wide, messaging between users whose interests match,
- use a persistent agent that runs most of the time, for long periods; the user does not start up the agent, get an immediate result, and shut it down, but instead runs it in the background for hours or weeks, while it uses 'word-of-mouth' to find and join appropriate groups of agents whose users share the same interests.

1.4 How the resulting clusters can be used

Once agents have formed clusters — an ongoing and continuous process for real agents on the Internet, due to the scale and constantly-changing environment involved —

how can these clusters be used? There are many applications — this is a short summary.

- Messaging into the group — a user whose agent is in some particular group can send a message into the group, either to those other agents known directly by the user's agent to be in the same cluster, or transitively through all other agents in the cluster by following cluster-cache information in a flooding algorithm. Thus, given some particular interest (represented by a local cluster of text files, and referred to as a granule later in this paper) on the user's local agent, the user could ask their agent to send a message to all other agents in the clump of which this interest is a member.
- Introductions — the chain of referrals themselves can be useful information, and can be exposed to the user under certain circumstances. Not only can the user send messages to particular individuals (whether pseudonymously or not), but the agent itself can facilitate a 'flirtatious' sort of introduction in which information can be symmetrically and gradually revealed, via cryptographic protocols.

Users could ask for an explicit introduction to particular members of the cluster, or could instruct their agent to accept or solicit introductions when it looked like there was a particularly good match available.

- Finding an expert — by using a combination of messaging into the group and introductions, the clusters in which a user's agent finds itself can potentially be used to find experts on the subject, since presumably such experts (if they, too, are represented by a Yenta agent) will have their interests reflected in the clustering. Here, a user could prepare a small piece of prose, or find some existing message, which talks about the subject for which the user wants an expert; the clustering algorithm could then generate a granule for this grain and attempt to find a suitable cluster. Once found, it could start the introduction process to acquaint the questioner and the expert.

1.5 What is described in this paper

The following sections describe the algorithm used in a prototype of the clustering system, the testing used to evaluate its performance, and how this work is integrated into the larger goal of automatically building interest groups and coalitions on the Internet.

2. The approach

The overall goal is to form clusters of agents whose users share similar interests. The details of the approach are covered in Foner [11] but essentially involve answering the following questions:

- What does it mean to have an interest, and how do agents know about these interests?
- How is similarity of interests determined?
- How does a particular agent know which other agents to contact?
- How are clusters of similar agents formed?

2.1 User interests

When an agent first starts running, it must determine what interests its user possesses and then update this from time-to-time to reflect changing interests. This could be achieved in a number of ways ranging from asking the user for keywords or phrases from time to time in order to gather the interests, or semi-automatically by analysis of the user's e-mail, documents or text files. The second approach has been used here for a number of reasons:

- the agent should be as unobtrusive as possible, therefore asking for keywords is not in line with this approach,
- users' interests could be defined in terms of a few keywords, but this would probably not give the right granularity of information — keywords such as agents, computing, music, etc, which would probably be adequate to give an overview of someone's interest areas would not be useful in the matchmaking sense of Yenta,
- using personal e-mail and documents provides a much richer set of words that can be used to refine different interest sets, and e-mail especially should reflect the gradually changing interests of a user,
- using personal information brings to the forefront issues of privacy and security which will rightly be an issue for the user.

Having taken this second approach of using personal data, specifically e-mail messages, the results of pre-clustering these messages into groups that represent distinct interests, as described in Foner [11], needs to be accurate. Although e-mails have been used to determine users' interests in this case, other information sources, such as documents or Web pages visited by the user, may provide additional information to determine more accurately the user's interests; however, these have not yet been explored.

The following section describes some of the results and issues with pre-clustering of e-mail messages.

2.2 *Effective pre-clustering*

A number of users were asked to submit quite large tracts of personal e-mail messages (between 1 and 20 megabytes worth each). These were clustered based on the contents of each message and the users were asked to mark the clusters using the following criteria:

- too specific,
- spot on,
- too general,
- garbage.

The initial clustering algorithm based on the whole e-mail message tended to produce a few very large clusters containing 50 to 100 e-mail messages, then many clusters with few messages — about 5, which had been specified as the minimum number of messages (grains) needed to form a cluster. In all, the algorithm would tend to produce 10 to 20 clusters from this data. Overall the results were quite poor, with only about 5 to 10% of the clusters being marked spot on and most of them being marked either garbage or too general.

Looking more closely at the resulting clusters, it became apparent that there were a number of flaws with this approach for the following reasons:

- some clusters were forming because words that formed part of a person's signature became dominant, and therefore, regardless of the content of the message, the signature became the clustering point,
- some long messages would cover a number of topics and would therefore get included in one cluster on the basis of one dominant paragraph,
- many e-mails were in the form of discussions covering a number of viewpoints and would therefore get included repeatedly.

To get round these problems some stringent pre-processing of e-mail messages was undertaken. This included partitioning long e-mail messages into paragraph-sized chunks. Removing any signatures, and removing any parts of 'included' messages. All of this was done automatically by a filter program.

Re-running the clustering on the preprocessed versions of the e-mails improved the markings of the spot-on clusters to between 20 to 25% from the previous 10% and produced

many more clusters to mark — typically clustering would produce about 50 clusters to mark.

Users were then asked to rank the spot-on clusters into the following categories:

- considered to be an expert in that area,
- would be interested in that area,
- would like to steer well clear of that area.

An interesting outcome of these markings was that most of the spot-on clusters that were marked 'steer-well-clear' were those clusters that contained meeting agendas or invitations!

2.3 *Data structures used in finding referrals and clusters*

The next step was where the various spot-on clusters (granules) in agents form clusters with other granules. For the sake of concreteness, assume that there are two agents, named *A* and *B*, which each have a few granules in them, e.g. *GA0*, *GA1*, etc. Each agent also contains several other data structures:

- a cluster cache (CC) which contains the names of all other agents currently known by some particular agent as being in the same cluster,
- a rumour cache (RC) which contains the names and other information (described below) of the last *r* agents with whom this agent has communicated,
- a pending-contact list (PC) which is a priority-ordered list of other agents that have been discovered but which the local agent has not yet contacted.

The rumour cache contains more than just the names of other agents encountered on the network. It also contains some subset, perhaps complete, of the text of each granule corresponding to those agents.

2.4 *Getting referrals and doing clustering*

Now that all this mechanism is in place, performing referrals and clustering is relatively uncomplicated [12, 13].

The process starts when some agent (call it *A*) has finished preclustering and has found at least one other agent (call it *B*). Agent *A* then performs a comparison of its local granules with those of agent *B*. When this comparison of granules from *A* with granules from *B* is complete, agent *A* may have found some acceptably close matches. Such matches are entered, one pair of granules at a time, in *A*'s cluster cache. *B* is likewise doing a comparison of its

granules with *A* and is entering items in its own cluster cache.

Whether or not any matches were found that were good enough to justify entering them in a cluster cache, the next step is to acquire referrals to agents that might be better matches. In the example here, agent *A* asks agent *B* for the entire contents of its rumour cache, and runs the same sort of comparison on those contents that it did on agent *B*'s own local granules. Good matches are added to *A*'s cluster cache, the rest of the data is added to *A*'s rumour cache, and *A*'s namelist is updated by adding to it those other agents which showed good matches to *A*, i.e. those agents which had granules that went into *A*'s cluster cache. These agents will be contacted next, after *A* finishes with *B* and any other entries in its namelist. The various caches belonging to *B* that *A* has been consulting were gathered by *B* in a similar way; every agent participating in this protocol is thus building up a collection of data for its own use and for the use of other agents.

This procedure acts somewhat like human word-of-mouth. If Sally asks Joe: "What should I look for in a new stereo?" Joe may respond: "I have no idea, but Alyson was talking to me recently about stereos and may know better." In effect, this has put Alyson into Sally's rumour cache (and, if Joe could quote something Alyson said that Sally found appropriate, perhaps into Sally's cluster cache as well). Sally now repeats the process with Alyson, essentially hill-climbing her way towards someone with the expertise to answer her question.

3. Experimental evaluation of the algorithm

The effectiveness of the algorithm has been explored in two ways. Firstly, an early simulation was run on a small number of agents (20) to see if it would work at all, and judge the overall efficiency of the algorithm. The results of this are reported in Foner [11].

There are a number of questions that needed answers to be sure the proposed algorithm would scale appropriately which were not addressed in the earlier experimental work. This section reports on a number of simulations that explore some of the issues with scalability and with tuning the various cache sizes.

Firstly, the earlier work only looked at 20 Yentas. With the intention of using this multi-agent system on a large scale, covering a whole organisation, or indeed the whole of the Internet population, probably the minimum grouping would be in the order of hundreds of Yentas. For smaller groups — say a few tens of Yentas — the matchmaking problems that Yenta is intended to address would probably not arise to make the system worthwhile. Clearly then the system must be tested with hundreds or thousands of Yentas.

There are a number of tuneable parameters with each Yenta such as the size of the rumour cache (i.e. the number of entries it can hold), the frequency at which Yentas will talk to others, the amount of data held from another Yenta, etc, which need to be set appropriately for the whole community of Yentas to work properly.

Clearly the frequency of Yentas talking to each other will be directly proportional to the time it will take to find all the other Yentas with similar interests (given that this approach works). A compromise therefore has to be reached on this level of 'chatter' between Yentas so that it is frequent enough for the Yentas to find out about all others with similar interests, but is not too frequent as to impair overall network traffic or CPU usage on the user's machine. It can probably be assumed that a person's interest is likely to be persistent for a few weeks, and certainly that their area of expertise would be likely to remain persistent for some extended time covering a few months. Given these time-scales, the Yentas can afford to be able to take some time over finding all the other Yentas with the same interest, and there would not be too much concern if it took a number of days of low-level communication for the Yentas to update fully their cluster caches.

The size of the rumour cache will have an impact on the way Yentas will find others with similar interests; the rumour cache holds information about other Yentas which are known about and some indication of their interests as a basis for deciding which Yentas to contact. A rumour cache that is very small would not give a Yenta any view as to which other ones to visit and would correspond to a very limited 'memory' which could cause the Yentas to repeatedly visit certain others. An unbounded or very large rumour cache on the other hand defeats the aim of the system, as every Yenta would get global (if not out-of-date) knowledge of all the others, which is counter-productive and probably unnecessary, as the view is that increasing the rumour cache above a particular size would probably give little benefit.

A final issue to be considered is how the interests are spread between Yentas. If the interests are spread very thinly between users, i.e. in the collection of Yentas only a very small percentage share the same (or similar) interest, it may take longer to cluster them rather than if there were few interests spread between the Yentas, as the cluster caches and rumour caches would contain many links to the others. Therefore it is important to determine what impact this would have on the effectiveness of the Yenta system.

3.1 Scope of the simulation

To explore these issues a number of simulations were run on a very simplified model of the Yenta system where

each Yenta was randomly given one interest out of those available. The impact of this was that the rumour cache and pending contact list were essentially the same thing. The simulations varied the following parameters:

- the number of Yentas in the simulation — this was varied between 200 and 1000,
- the size of the rumour cache, which varied between 2 and 50 entries,
- the number of different interests spread between Yentas, which varied between 5 and 30.

In all the simulations, each Yenta was allowed 500 message exchanges with other Yentas. After every 10 message exchanges, data was collected on the size of each Yenta's cluster cache. This information was then processed to produce a number of animations which were then analysed. The following 3-D graphs (Figs 1—3) are the mid and final frame of each of these animations. In all the graphs, the height of the columns shows the overall effectiveness of the Yenta system as a percentage — 100% indicates that all of the Yentas have found all their counterparts with the same interest ¹. The back walls on the graphs depict the minimum, average and maximum values for the corresponding row of columns.

3.2 Does it scale up?

One of the first questions to be answered was does it scale up to hundreds or thousands of Yentas. The answer to that is certainly yes. Taking Fig 1 which shows the coverage of Yentas after 500 iterations where the number of Yentas is varied from 200 to 1000, and the number of distinct interests is varied from 5 to 30, it is seen that, in almost all cases, greater than 90% coverage has been achieved, and, for low number of interests, 100% coverage. Also, from that figure, there can be a reasonable level of confidence that it will scale to greater numbers of Yentas, because as the number of Yentas is increased, the coverage does not get appreciably worse. However, there is an interesting dependency as the number of distinct interests is increased, which deserves more exploration.

3.3 How long does it take?

These simulations were limited to 500 message exchanges between Yentas. This figure of 500 came about from some preliminary experiments to determine how many exchanges were needed; however, it can be seen from the graphs that, as not all Yentas have converged by this time,

¹The percentage is the average of each of the Yenta's coverage. As the standard deviation was also measured but in all cases was very low (no more than four percent), it has been omitted.

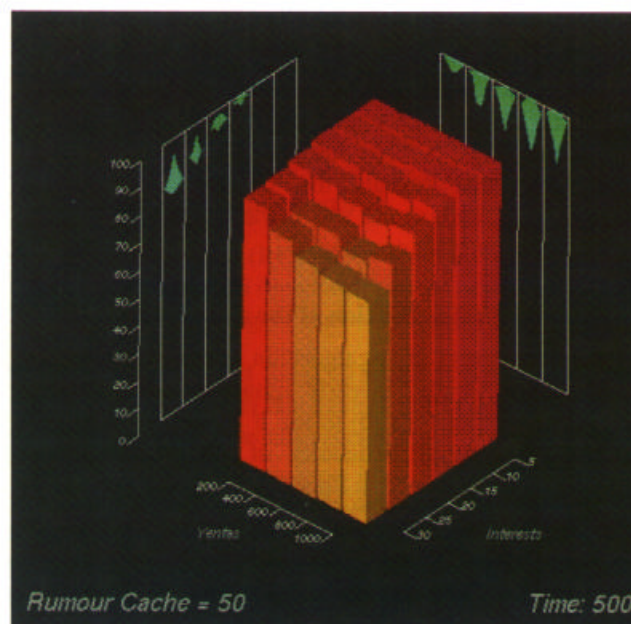
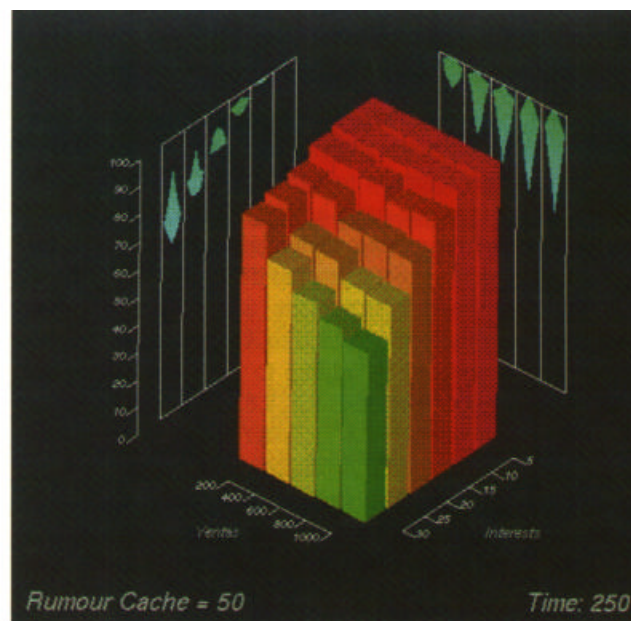


Fig 1 Varying the number of interests and number of Yentas.

for larger number of Yentas more time will probably be necessary. As discussed earlier, it is important to get the balance right to ensure the Yentas find each other yet do not cripple their hosts in the process. Therefore, if a moderate communications scheme is suggested where a Yenta communicates every 10 minutes, that gives 500 exchanges in 83 hours or about three and a half days. This is well within the expected window of a user having an area of expertise for a minimum of weeks or months. As before, Fig 1 shows that the majority of Yentas have found all their peers in this time.

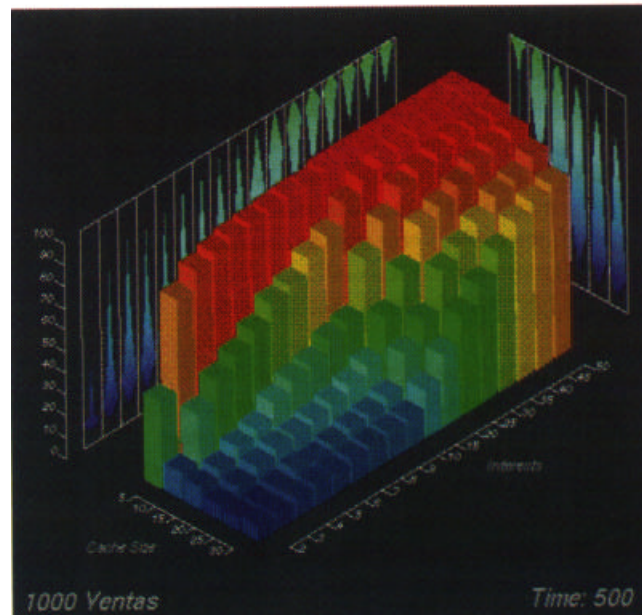
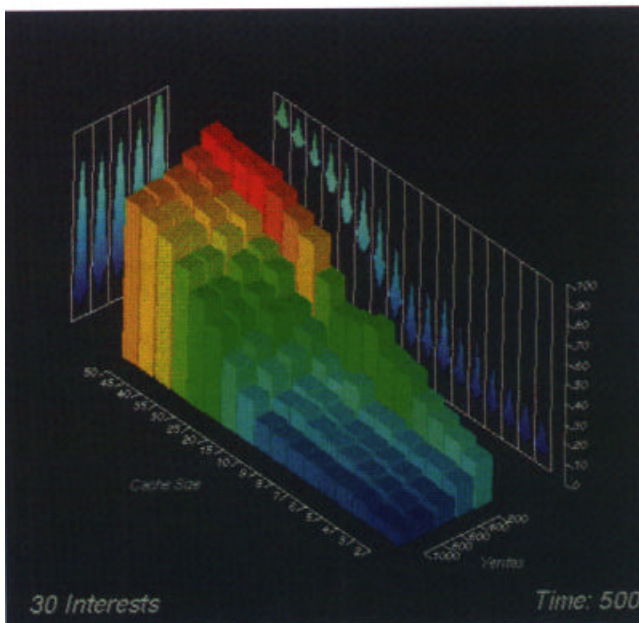
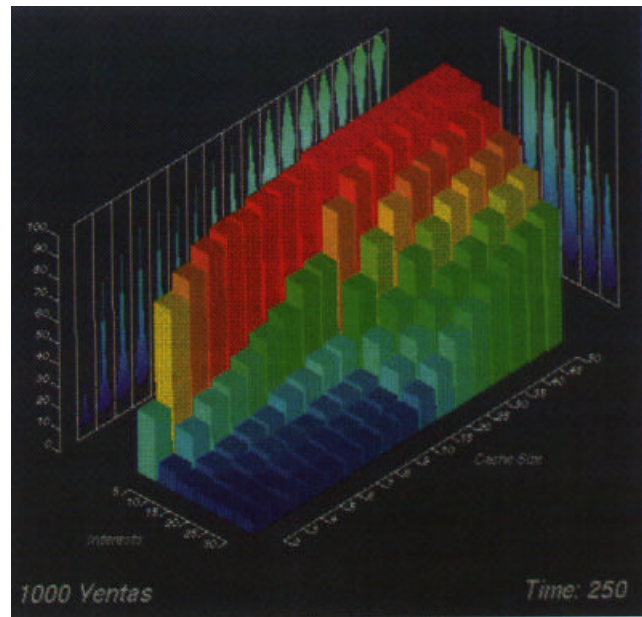
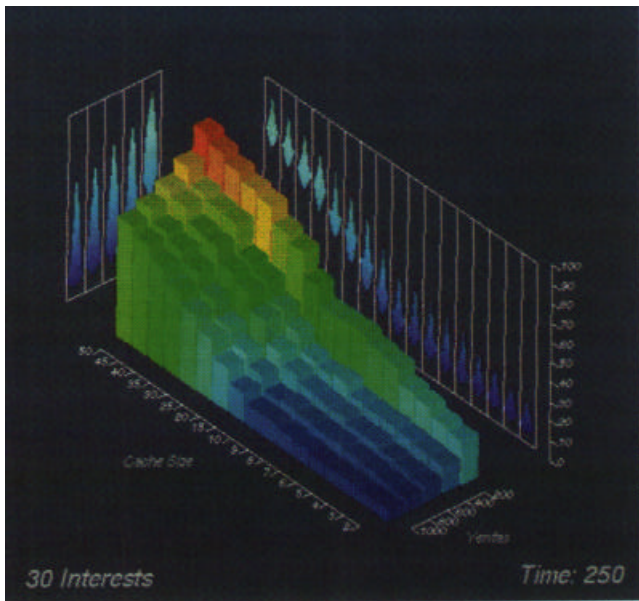


Fig 2 Varying the rumour cache size and number of Yentas.

Fig 3 Varying the number of interests and rumour cache size.

3.4 What is the impact of the rumour cache size?

It was suggested that there might be minimum and maximum useful rumour cache sizes and the rumour cache size was varied in the experiments to see if this was borne out. Figure 2 shows quite clearly that as the size of the rumour cache is increased, much better results for the system as a whole are obtained when the cache is 30 or above, while a rumour cache with 10 entries or less performs very poorly even with very few Yentas. It can be seen, therefore, from the data in Fig 2, that the cache size needs to be over 30. Taking Fig 3 now, where the rumour

cache is varied against the number of different interests, there is certainly a dependency with increasingly varied interests.

It appears that with high numbers of distinct interests a rumour cache of 50 is still too small, whereas with a small number of distinct interests there is no improvement once the rumour cache size is greater than 5!

3.5 Further questions

Scaling issues for up to 1000 Yentas have been investigated. Although this is a reasonable number for a

system to work with in a small-to-medium-sized company, it does not get anywhere close to the size of the Internet community, for which it was originally intended. It can be seen from the results that, with large numbers of different interests, as the number of Yentas is increased to 1000, not all Yentas (about 15%) have been updated after 500 iterations, and this would get worse as the number of Yentas is increased still further. It does appear though that it would scale acceptably to approximately 1500 without any changes, and would improve if the size of the rumour cache were increased. From this work, it cannot be predicted how the system would perform with tens or hundreds of thousands of Yentas and more experiments would need to be undertaken to determine its performance.

4. Related work

There are many experiments in distributed AI and multi-agent systems which could be considered relevant; here only other matchmaking systems and related approaches are considered.

A common technique in systems that support computation among a group of users is to centralise a server and have its users act like clients. Systems that match user interests to each other, and have such a centralised structure, include Webhound [7] and HOMR/Ringo [1].

Kuokka and Harada [14] describe a system that matches advertisements and requests from users and hence serves as a brokering service. Their system certainly is a matchmaker, but it assumes a centralised matchmaker and a highly structured representation of user interests.

Others have taken a more distributed approach. For example, Kautz, Milewski and Selman [15] report work on a prototype system for expertise location in a large company. Their prototype assumes that users can identify who else might be a suitable contact, and uses agents to automate the referral-chaining process; they include simulated results showing how the length and accuracy of the resulting referral chains are affected by the number of simulated users and the accuracy and helpfulness of their recommendations. Yenta differs from this approach in using ubiquitous user data to infer interests, rather than explicitly asking about expertise.

5. Conclusions and future work

These results demonstrate that referral-based matchmaking can provide acceptable results without requiring any one agent to know about all other agents, and without requiring unreasonable messaging traffic or local computation.

Work is currently proceeding on several aspects of the final Yenta design:

- implementing the requisite privacy safeguards and user interface to permit a networked implementation with real user data,
- experimenting with different comparison metrics to enhance Yenta's ability to determine accurately a match in user interests,
- experimenting with the prototype with agents representing users from both BT and MIT Media Lab.

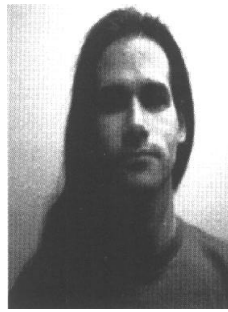
6. Acknowledgments

The authors would like to thank undergraduates Jon Litt, for figuring out the intricacies of a large system like SMART [16] and for his systems administration expertise, Bayard Wenzel for implementing the initial prototype of these ideas, and Simon Greatrix of BT Laboratories for creating the animations.

References

- 1 Lashkari Y, Metral M and Maes P: 'Collaborative Interface Agents,' Proceedings of the Twelfth National Conference on Artificial Intelligence, MIT Press, Cambridge, MA (1994).
- 2 Maes P: 'Agents that Reduce Work and Information Overload', Communications of the ACM, 37, No 7 (July 1994).
- 3 Kozierok R and Maes P: 'A Learning Interface Agent for Meeting Scheduling,' Proceedings of the 1993 International Workshop on Intelligent User Interfaces, ACM Press, New York (1993).
- 4 Foner L N: 'What's an Agent, Anyway? A Sociological Case Study,' Agents Memo 93-01, MIT Media Lab, Cambridge, MA, <http://foner.www.media.mit.edu/people/foner/Julia/> (1993).
- 5 Mauldin M: 'ChatterBots, TinyMuds, and the Turing Test: Entering the Loebner Prize Competition,' Proceedings of the Twelfth National Conference on Artificial Intelligence, MIT Press, Cambridge, MA, <http://fuzine.mc.cs.cmu.edu/mlm/aaai94.htm> (1994).
- 6 Fisk D: 'Application of Social Filtering to Movie Recommendation', BT Technol J, 14, No 4, pp 124—132 (October 1996).
- 7 Shardanand U and Maes P: 'Social Information Filtering: Algorithms for Automating Word of Mouth', Proceedings of the CHI '95 Conference (1995).
- 8 Mockapetris P: 'Domain names — implementation and specification,' Internet RFC1034 (1987), {<http://fuzine.mc.cs.cmu.edu/mlm/aaai94.html>}

- 9 Horton M and Adams R: 'Standard for interchange of USENET messages,' Internet RFC1036, <ftp://ds.internic.net/rfc/rfc1036.txt> (1987).
- 10 Huberman B A (Ed): 'The Ecology of Computation', Elsevier Science Publishers B V (1988).
- 11 Foner L N: 'A Multi-Agent Referral System for Matchmaking', PAAM'96 Proceedings, pp 245—262 (April 1996).
- 12 Foner L N: 'Clustering and Information Sharing in an Ecology of Cooperating Agents, or How to Gossip without Spilling the Beans,' Proceedings of the Conference on Computers, Freedom, and Privacy, Student Paper Winner, Burlingame, CA (1995).
- 13 Foner L N: 'Clustering and Information Sharing in an Ecology of Cooperating Agents,' AAAI '95 Spring Symposium Workshop Notes on Information Gathering in Distributed, Heterogeneous Environments, Stanford, CA, <http://www.isi.edu/sims/knoblock/sss95/foner.ps> (1995).
- 14 Kuokka D and Harada L: 'Matchmaking for Information Agents,' Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI) (1995).
- 15 Kautz H, Milewski A and Selman B: 'Agent Amplified Communication,' AAAI '95 Spring Symposium Workshop Notes on Information Gathering in Distributed, Heterogeneous Environments, Stanford, CA (1995).
- 16 Zumoff J: 'Users Manual for the SMART Information Retrieval System,' Cornell Technical Report 71—95 (1995).



Leonard N Foner, a PhD candidate in the Agents Group of the MIT Media Lab, is creating Yenta, a distributed, multi-agent matchmaker, which finds people with similar interests on the Internet and builds coalitions and communities via careful, privacy-protecting introductions. His research is part of a collaboration between BT Laboratories and the MIT Media Lab. He has interests in AI, networking, cryptography, and sociology, and recently served on the committee of the 1996 Conference on Computers, Freedom, and Privacy. His SM thesis studied agents, machine learning, and focus of attention.

Before going to the Media Lab, he spent three years each at Bitstream, designing expert systems which helped graphic artists make beautiful fonts, and at Symbolics, as a Lisp Machine developer. His Bachelor's thesis, from the MIT AI Lab, was an expert system that debugged ailing networks.



Barry Crabtree joined BT in 1980 after completing a Physics degree at Bath University. After a number of years in software development he moved into the field of artificial intelligence (AI) and helped begin BT's systems in automated diagnosis and workforce management. He then moved on to work in distributed AI, optimisation problems and adaptive systems. He is currently a Technical Advisor in the Advanced Applications and Technology department, and has a particular interest in intelligent software agents.